

C# FUNDAMENTALS — DEEP & SIMPLE NOTES

Beginner → Interview Ready | Why each thing is written, what it does, and when to use it

Goal: Build a strong C# foundation so that topics like ASP.NET Core, Web API, EF Core, LINQ, Dependency Injection, CQRS and Clean Architecture become much easier.

How to study: Read the explanation → type the example yourself → change the example → predict the output → run it.

CONTENTS

1. What is C# and .NET?
2. Program structure and Main method
3. Variables, data types and memory basics
4. Type conversion, parsing and casting
5. Operators
6. Input and output
7. if / else / switch
8. Loops
9. Methods, parameters, return values and overloading
10. Arrays, strings and StringBuilder
11. Value types vs reference types
12. Classes and objects
13. Constructors, this and object initialization
14. Encapsulation and access modifiers
15. Properties, fields and auto-properties
16. static
17. Inheritance
18. Polymorphism, virtual, override and new
19. Abstraction and abstract classes
20. Interfaces
21. enum and struct
22. Nullable types and null
23. Exception handling
24. Collections: List, Dictionary, HashSet, Queue, Stack
25. Generics
26. Delegates, Action, Func and Predicate
27. Lambda expressions
28. LINQ fundamentals

- 29. var, const and readonly
- 30. async / await and Task basics
- 31. Important C# interview traps
- 32. Mini practice set + roadmap

1. What is C# and .NET?

C# (pronounced C-sharp) is a programming language created by Microsoft. It is used to build Web APIs, backend systems, desktop applications, cloud services, games and more.

.NET is the platform/runtime/ecosystem that runs C# programs and provides libraries for things such as collections, files, networking, HTTP, JSON and database access.

Think of it like this:

C# = the language you write.

.NET = the platform + runtime + libraries that execute and support your C# code.

Example:

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Hello");
    }
}
```

Why `using System;`? It lets us use types from the System namespace without writing `System.Console` every time.

Why `class Program`? A class is a type that can contain methods and data.

Why `static void Main()`? Main is the entry point of a traditional console application. static means it can be called without creating a Program object; void means it returns no value.

Why `Console.WriteLine`? Console is a .NET class and WriteLine prints text.

2. Program structure and Main method

A C# program is made from namespaces, types such as classes/structs, and members such as methods/properties/fields.

Modern C# also supports top-level statements, so you may see:

```
Console.WriteLine("Hello");
```

This is shorter because the compiler creates the required program entry-point structure for you. In enterprise projects you will often still encounter explicit classes and methods, so understand both styles.

A method has a name, parameters, a return type, and a body:

```
static int Add(int a, int b)
{
    return a + b;
}
```

int before Add = return type. a, b = inputs/parameters. return sends a value back to the caller.

3. Variables, data types and memory basics

A variable is a named location used to hold a value.

Common C# built-in types:

int → whole numbers

long → larger whole numbers

double → floating-point numbers
decimal → high-precision decimal values, commonly useful for money
bool → true/false
char → one character
string → text

Example:

```
int age = 26;  
double height = 6.0;  
decimal salary = 19000.50m;  
bool isActive = true;  
char grade = 'A';  
string name = "Saurabh";
```

Why the `m` after 19000.50? It tells C# that the literal is a decimal. Without it, a decimal-looking literal is normally treated as double.

Important distinction:

```
<b>Declaration:</b> int age;  
<b>Initialization:</b> int age = 26;  
<b>Assignment:</b> age = 27;
```

A variable's type controls what kind of value can be stored and what operations are valid.

4. Type conversion, parsing and casting

C# is strongly typed, so converting between types must follow rules.

Implicit conversion: usually safe conversion where the destination can represent all source values.

```
int x = 10;  
long y = x; // implicit conversion
```

Explicit cast: you tell C# to convert:

```
double d = 10.8;  
int n = (int)d; // n becomes 10
```

The fractional part is discarded; this is not rounding.

Parsing: converting text into a value:

```
string text = "123";  
int number = int.Parse(text);
```

If the text may be invalid, TryParse is safer:

```
if (int.TryParse("123", out int number))  
{  
    Console.WriteLine(number);  
}
```

Why TryParse? It avoids throwing an exception for normal invalid user input such as "abc".

5. Operators

Operators perform actions on values.

Arithmetic: + - * / %

Comparison: == != > < >= <=

Logical: && || !

Assignment: = += -= *= /=

Null-related: ?? and ?.

```
int a = 10;
int b = 3;

Console.WriteLine(a + b); // 13
Console.WriteLine(a % b); // 1
Console.WriteLine(a > b); // True

bool result = a > 5 && b < 5; // True
```

% gives the remainder. It is very useful for even/odd checks:
number % 2 == 0.

&& means AND: both conditions must be true.

|| means OR: at least one must be true.

! negates a boolean.

6. Input and output

Console.WriteLine prints and moves to the next line. Console.Write prints without automatically moving to the next line.

Console.ReadLine reads a line of text. Since input is text, convert it when you need a number.

```
Console.Write("Enter age: ");
string? input = Console.ReadLine();

if (int.TryParse(input, out int age))
{
    Console.WriteLine($"Age = {age}");
}
```

The \$ before a string enables string interpolation, so {age} is replaced by the variable value. The ? on string? communicates that the value may be null.

7. if / else / switch

Use if when a decision depends on a condition.

```
int marks = 82;

if (marks >= 90)
{
    Console.WriteLine("A");
}
else if (marks >= 60)
{
    Console.WriteLine("B");
}
else
{
    Console.WriteLine("C");
}
```

switch is useful when comparing one value against several patterns/cases:

```

string role = "Admin";

switch (role)
{
case "Admin":
    Console.WriteLine("Full access");
    break;

case "User":
    Console.WriteLine("Limited access");
    break;

default:
    Console.WriteLine("Unknown");
    break;
}

```

Why break? In a traditional switch section it ends that case. Modern switch expressions can be shorter, but first master the normal switch.

8. Loops

Loops repeat code.

for is useful when you know/control the number of iterations.

```

for (int i = 0; i < 5; i++)
{
    Console.WriteLine(i);
}

```

Read it as: start i at 0; continue while i < 5; after each iteration execute i++.

foreach is ideal for visiting every item:

```

int[] numbers = { 10, 20, 30 };

foreach (int number in numbers)
{
    Console.WriteLine(number);
}

```

while repeats while a condition is true. **do-while** runs the body at least once.

Be careful with loop conditions; a condition that never becomes false can create an infinite loop.

9. Methods, parameters, return values and overloading

Methods package reusable logic.

```

static int Add(int a, int b)
{
    return a + b;
}

int result = Add(10, 20);

```

Why use a method? To avoid repeating logic, improve readability, testing and maintenance.

Parameters are inputs. The return type tells what comes back.

void means the method does not return a value:

```
static void PrintName(string name)
{
    Console.WriteLine(name);
}
```

Method overloading means same method name but different parameter lists:

```
static int Add(int a, int b) => a + b;
static int Add(int a, int b, int c) => a + b + c;
```

The compiler chooses the matching overload from the arguments you pass.

10. Arrays, strings and StringBuilder

An array stores multiple values of the same type and has a fixed length.

```
int[] nums = { 10, 20, 30 };
```

```
Console.WriteLine(nums[0]); // 10
Console.WriteLine(nums.Length); // 3
```

Indexes start at 0. `nums[0]` is the first item.

A string represents text:

```
string firstName = "Saurabh";
string fullName = firstName + " Yadav";
Console.WriteLine(fullName.Length);
```

Strings are immutable: operations that appear to change a string generally create a new string.

For many repeated modifications, `StringBuilder` can be more efficient:

```
var sb = new System.Text.StringBuilder();
sb.Append("Hello");
sb.Append(" ");
sb.Append("World");

string result = sb.ToString();
```

Use `StringBuilder` when you are repeatedly building/modifying text, especially in loops.

11. Value types vs reference types

This is one of the most important C# concepts.

Common value types: `int`, `double`, `bool`, `char`, `struct`, `enum`.

Common reference types: `class`, `string`, `array`, `delegate`.

Conceptually, a value-type variable contains its value. A reference-type variable contains a reference to an object.

```
int a = 10;
int b = a;
b = 20;
// a is still 10

Person p1 = new Person();
p1.Name = "A";

Person p2 = p1;
p2.Name = "B";
// p1.Name is now "B" too
```

Why? With the object example, p1 and p2 refer to the same Person object. Changing the object through one reference is visible through the other.

This is why copying a reference-type variable is not the same as copying the entire object.

12. Classes and objects

A class is a blueprint/type. An object is an instance created from that class.

```
class Car
{
    public string Brand;
    public void Start()
    {
        Console.WriteLine("Car started");
    }
}

Car car = new Car();
car.Brand = "Toyota";
car.Start();
```

Why `<b>new</b>`? It creates an object instance.

Think:

Class = blueprint

Object = actual house built from the blueprint.

A class can contain fields, properties, methods, constructors, events and more.

13. Constructors, this and object initialization

A constructor runs when an object is created. It is commonly used to put an object into a valid initial state.

```
class Employee
{
    public string Name;
    public int Age;

    public Employee(string name, int age)
    {
        this.Name = name;
        this.Age = age;
    }
}
```

```
var e = new Employee("Saurabh", 26);
```

this means the current object. It is useful when parameter names and field/property names are the same.

You can also use object initializers:

```
var e = new Employee
{
    Name = "Saurabh",
    Age = 26
};
```

Object initializers set accessible properties/fields after construction. Use constructors when the object requires mandatory initialization or validation.

14. Encapsulation and access modifiers

Encapsulation means controlling how an object's internal state is accessed or changed.

Main access modifiers:

public → accessible from anywhere allowed by the type

private → only inside the containing type

protected → containing type and derived types

internal → same assembly

protected internal / private protected → combinations with assembly/inheritance rules

Example:

```
class BankAccount
{
    private decimal balance;

    public void Deposit(decimal amount)
    {
        if (amount > 0)
            balance += amount;
    }

    public decimal GetBalance()
    {
        return balance;
    }
}
```

Why make balance private? So outside code cannot freely do balance = -100000. The class controls valid changes through methods.

This protects invariants and makes code easier to maintain.

15. Properties, fields and auto-properties

A field directly stores data inside an object. A property provides a controlled way to expose data.

```
class Employee
{
    private string name;
```

```
public string Name
{
    get { return name; }
    set { name = value; }
}
```

Modern C# commonly uses auto-properties:

```
public string Name { get; set; }
public int Age { get; private set; }
```

get; set; means callers can read and write.

get; private set; means callers can read, but only the class can change the value.

Properties are heavily used in ASP.NET Core models, DTOs and EF Core entities.

16. static

static means the member belongs to the type itself rather than to each individual object.

```
class MathHelper
{
    public static int Square(int x)
    {
        return x * x;
    }
}

int result = MathHelper.Square(5);
```

No MathHelper object is needed because Square is static.

Static fields are shared by the type:

```
class Counter
{
    public static int Count = 0;
}
```

Use static when the behavior/data conceptually belongs to the type and does not require per-object state. Do not make everything static; instance objects are usually better when state differs per object.

17. Inheritance

Inheritance lets a derived class reuse/extend a base class.

```
class Animal
{
    public void Eat()
    {
        Console.WriteLine("Eating");
    }
}

class Dog : Animal
{
    public void Bark()
```

```

{
Console.WriteLine("Barking");
}
}

var dog = new Dog();
dog.Eat();
dog.Bark();

```

Dog : Animal means Dog derives from Animal. Dog gets accessible members of Animal and can add its own behavior.

Inheritance models an IS-A relationship: Dog IS-A Animal.

18. Polymorphism, virtual, override and new

Polymorphism means the same base-type reference can work with different derived implementations.

```

class Animal
{
public virtual void Speak()
{
Console.WriteLine("Animal sound");
}
}

class Dog : Animal
{
public override void Speak()
{
Console.WriteLine("Bark");
}
}

Animal a = new Dog();
a.Speak(); // Bark

```

Why **virtual**? It allows a derived class to replace the behavior.

Why **override**? It says the derived class is providing the overridden implementation.

new is different: it hides a base member rather than participating in virtual dispatch. This distinction matters in interviews and real code.

19. Abstraction and abstract classes

Abstraction means exposing what an object must do while hiding implementation details.

```

abstract class Payment
{
public abstract void Pay(decimal amount);

public void PrintReceipt()
{
Console.WriteLine("Receipt");
}
}

```

```

class UpiPayment : Payment
{
public override void Pay(decimal amount)
{
Console.WriteLine($"Paid {amount}");
}
}

```

An abstract class cannot normally be instantiated directly. An abstract method has no implementation in the base class and forces derived classes to provide one.

Use an abstract class when related classes share common state/behavior plus some required variation.

20. Interfaces

An interface defines a contract: a type that implements it agrees to provide the required members.

```

interface INotification
{
void Send(string message);
}

class EmailNotification : INotification
{
public void Send(string message)
{
Console.WriteLine("Email: " + message);
}
}

```

Why use an interface? It reduces coupling. Code can depend on INotification instead of knowing the concrete EmailNotification class.

This is very important in ASP.NET Core Dependency Injection:

```

public class OrderService
{
private readonly INotification _notification;

public OrderService(INotification notification)
{
_notification = notification;
}
}

```

The service depends on the abstraction (interface), while the application can supply a concrete implementation.

21. enum and struct

enum represents a named set of integral values.

```

enum OrderStatus
{
Pending,
Paid,
Shipped,
}

```

```
Cancelled
}
```

```
OrderStatus status = OrderStatus.Paid;
```

Enums improve readability compared with magic numbers such as 0, 1, 2.

A struct is a value type, often used for small values with value semantics:

```
struct Point
{
    public int X;
    public int Y;
}
```

```
Point p = new Point { X = 10, Y = 20 };
```

Structs can be useful for small data values. Classes are reference types and are generally preferred for entities with identity and more complex mutable behavior.

22. Nullable types and null

null means a reference does not currently refer to an object. For nullable value types, it represents the absence of a value.

```
int? age = null;

if (age.HasValue)
{
    Console.WriteLine(age.Value);
}
```

The ? makes a value type nullable.

For reference types, nullable reference type annotations help the compiler warn about possible nulls:

```
string? name = null;

string displayName = name ?? "Unknown";
```

?? means: if the left side is null, use the right side.

?. is the null-conditional operator:

```
int? length = name?.Length;
```

This prevents a `NullReferenceException` when name is null.

23. Exception handling

Exceptions represent unexpected/error conditions that interrupt normal execution.

```
try
{
    int x = int.Parse("abc");
}
catch (FormatException ex)
{
}
```

```
Console.WriteLine("Invalid number");
}
finally
{
Console.WriteLine("Finished");
}
```

try contains risky code.

catch handles a matching exception.

finally runs whether an exception occurred or not (subject to abnormal process termination).

Do not use exceptions as normal control flow when TryParse or validation is appropriate. Catch specific exceptions when possible.

24. Collections

Arrays have fixed size. Generic collections are more flexible.

List: ordered, dynamically sized collection.

Dictionary: key → value lookup.

HashSet: unique values.

Queue: first-in-first-out.

Stack: last-in-first-out.

```
var names = new List<string>();
names.Add("A");
names.Add("B");

var ages = new Dictionary<string, int>();
ages["A"] = 26;

var unique = new HashSet<int> { 1, 2, 2, 3 };

var queue = new Queue<string>();
queue.Enqueue("first");
string item = queue.Dequeue();
```

The T is the element type. List means a list whose elements are strings.

Use Dictionary when you frequently need to find a value by a key. Use HashSet when uniqueness/membership is the main concern.

25. Generics

Generics let you write reusable, type-safe code without hard-coding one data type.

```
class Box<T>
{
public T Value { get; set; }
}

var intBox = new Box<int>();
intBox.Value = 10;

var stringBox = new Box<string>();
stringBox.Value = "Hello";
```

Here T is a placeholder for a type. When you create Box<int>, T becomes int.

Generics avoid unnecessary casts and reduce duplicated code. List<T>, Dictionary<TKey,TValue>, Task<T> and many .NET APIs use generics.

26. Delegates, Action, Func and Predicate

A delegate is a type-safe reference to a method.

```
delegate int Operation(int a, int b);

static int Add(int a, int b) => a + b;
```

```
Operation op = Add;
Console.WriteLine(op(2, 3));
```

Built-in delegates are common:

Action → returns void.

Func<T> → returns a value; last generic type is the return type.

Predicate<T> → returns bool.

```
Action<string> print = name => Console.WriteLine(name);
```

```
Func<int, int, int> add = (a, b) => a + b;
```

```
Predicate<int> isEven = n => n % 2 == 0;
```

Delegates are heavily used with events, LINQ and callbacks.

27. Lambda expressions

A lambda is a short way to write a function expression.

```
Func<int, int> square = x => x * x;
Console.WriteLine(square(5));
```

Read `x => x * x` as: take x and return `x*x`.

With multiple parameters:

```
Func add = (a, b) => a + b;
```

Lambdas become especially important in LINQ:

```
var adults = people.Where(p => p.Age >= 18);
```

The expression `p => p.Age >= 18` is the condition used by Where.

28. LINQ fundamentals

LINQ (Language Integrated Query) provides a consistent way to query collections and other data sources.

Common methods:

Where → filter

Select → transform/project

OrderBy → sort

FirstOrDefault → first item or default

Any → whether at least one item matches

Count → number of items

GroupBy → group items

```
var numbers = new List<int> { 1, 2, 3, 4, 5, 6 };

var even = numbers
    .Where(n => n % 2 == 0)
    .Select(n => n * 10)
    .ToList();
```

Step 1: Where keeps 2,4,6.

Step 2: Select converts them to 20,40,60.

Step 3: ToList executes/materializes the result into a List.

A major interview point: many LINQ operators use deferred execution. The query may not run until you enumerate it or materialize it with methods such as ToList.

29. var, const and readonly

var means the compiler infers the variable's static type from the assigned expression. It does NOT mean dynamically typed.

```
var age = 26; // int
var name = "Saurabh"; // string
```

You still cannot later assign a string to age.

const is a compile-time constant:

```
const double Pi = 3.141592653589793;
```

A const must be known at compile time.

readonly fields can be assigned at declaration or in a constructor and then cannot be reassigned:

```
class Config
{
    private readonly string connectionName;

    public Config(string name)
    {
        connectionName = name;
    }
}
```

Use const for true compile-time constants. Use readonly when a value is fixed after object construction.

30. async / await and Task basics

Async programming allows an operation that waits for something (for example I/O) to avoid blocking the current thread while it waits.

```
public async Task<string> GetDataAsync()
{
```

```
string result = await SomeloOperationAsync();
return result;
}
```

`Task<string>` represents an asynchronous operation that will eventually produce a string.
`async` enables `await` inside the method.
`await` asynchronously waits for the Task to complete and then continues.

A common mistake is thinking `async` automatically creates a new thread. It does not. For I/O-bound work, `async/await` mainly helps free the thread while the I/O is in progress.

31. Important C# interview traps

1. `==` vs `Equals()`: `==` may be overloaded; `Equals` is a method whose behavior depends on the type.
2. `string` is immutable: repeated concatenation can create many strings.
3. `List` vs `Array`: `List` grows dynamically; array length is fixed after creation.
4. `class` vs `struct`: `class` is reference type; `struct` is value type.
5. `ref` vs `out`: both pass by reference; `out` does not need to be definitely assigned before the call, and the called method must assign it.
6. `interface` vs `abstract class`: `interface` is primarily a contract; `abstract class` can provide shared state/implementation.
7. `virtual/override` vs `new`: `override` participates in virtual dispatch; `new` hides a member.
8. `throw` vs `throw ex`: inside a catch, `throw` preserves the original stack trace; `throw ex` resets the stack trace location.
9. `IEnumerable` vs `List`: `IEnumerable` represents enumeration; `List` is a concrete collection with storage and additional operations.
10. `deferred LINQ`: `Where/Select` often don't execute until enumeration.
11. `async void`: generally avoid except event handlers; prefer `Task/Task<T>` for `async` methods.
12. `null safety`: nullable annotations help detect possible null problems at compile time but do not magically prevent all nulls at runtime.

32. Mini practice set + roadmap

Practice these in order:

Level 1 — Fundamentals

1. Reverse a number.
2. Check even/odd.
3. Find maximum of three numbers.
4. Print 1 to N.
5. Sum an array.
6. Count vowels in a string.

Level 2 — OOP

7. Create `Employee` class with private fields and properties.
8. Create `BankAccount` with `Deposit/Withdraw` validation.
9. Create `Animal` → `Dog/Cat` using `virtual/override`.
10. Create `INotification` and `Email/SMS` implementations.

Level 3 — Collections/LINQ

11. Find duplicate numbers using `HashSet`.
12. Count word frequency using `Dictionary`.
13. Filter employees by salary using `LINQ`.
14. Group employees by department.
15. Sort employees by salary descending.

Level 4 — Backend connection

Understand these after fundamentals:

C# → OOP → Generics → Collections → Delegates/Lambda → LINQ → async/await → ASP.NET Core → Web API → DI → EF Core → Authentication/JWT → Clean Architecture/CQRS.

Golden rule: whenever you see syntax you don't understand, ask three questions:

1. What type is this?
2. What value/state does it represent?
3. Why is this keyword/operator needed here?

Example:

private readonly INotification _notification;

private = who can access it?

readonly = can the field reference be reassigned after construction?

INotification = what contract/type does it depend on?

_notification = variable/field name.

QUICK CHEAT SHEET

Data: int, long, double, decimal, bool, char, string

Decision: if/else, switch

Repeat: for, foreach, while, do-while

Reuse: methods, classes, generics

OOP: encapsulation, inheritance, polymorphism, abstraction

Contracts: interfaces

Data structures: array, List, Dictionary, HashSet, Queue, Stack

Functional style: delegate, Action, Func, Predicate, lambda, LINQ

Errors: try/catch/finally

Async: Task, async, await

Null: ?, ??, ?.

FINAL ADVICE

Do not try to memorize C# syntax line-by-line. Understand the reason behind the syntax and write small programs. For backend interviews, spend extra time on OOP, collections, generics, delegates/lambdas, LINQ, exception handling, async/await, interfaces and dependency injection.